Prepare answers before the exercise class. In class you will present them.

2.1 Does the following statement satisfy the synchronous assumption?

"On my server, no request ever takes more than 1 week to be processed."

2.2 Models in distributed systems

a) T F (T=true/F=false)

- ☐ ☐ A model is accurate only if it is tractable
- ☐ ☐ Models of distributed systems make it possible to predict the feasibility and costs of algorithms or protocols
- ☐ ☐ In an asynchronous system you make the assumption that for most messages the time between sending and receiving the message is infinite
- ☐ ☐ Using a Byzantine fault model instead of a crash fault model has only advantages
- ☐ ☐ Algorithms designed for an asynchronous system model can be used without modification in a synchronous system.

b) Can you describe a case (some specific assumptions) in which the crash-recovery and the fail-silent models are practically identical?

2.3 Properties of logical clocks

The relation "$\mathfrak{a}$ happens causally before $\mathfrak{b}$" is written as $\mathfrak{a} \rightarrow \mathfrak{b}$. Two events, $\mathfrak{a}$ and $\mathfrak{b}$, have known logical (Lamport) time stamps $C(\mathfrak{a})$ and $C(\mathfrak{b})$, such that $C(\mathfrak{a}) < C(\mathfrak{b})$.

a) What can you say about the correctness of the following statements (certainly true/certainly false/undecidable)?

- $\mathfrak{a} \rightarrow \mathfrak{b}$
- $\mathfrak{b} \rightarrow \mathfrak{a}$
- $\mathfrak{a}$ happened before $\mathfrak{b}$ in real time
- $\mathfrak{b}$ happened before $\mathfrak{a}$ in real time
- $\mathfrak{a}$ and $\mathfrak{b}$ are concurrent (i.e., neither $\mathfrak{a} \rightarrow \mathfrak{b}$ nor $\mathfrak{b} \rightarrow \mathfrak{a}$ is true)

b) What can you say about these statements, if you replace the time stamps with values of a vector clock $V(\mathfrak{a})$ and $V(\mathfrak{b})$, and again assume $V(\mathfrak{a}) < V(\mathfrak{b})$?

2.4 Logical clocks and fault tolerance

What effect can a crash of a single process have on Lamport logical clocks with respect to the basic ordering property of the algorithm? (i.e., is it possible that due to a crash the order property $\mathfrak{a} \rightarrow \mathfrak{b} \Longrightarrow C(\mathfrak{a}) < C(\mathfrak{b})$ is violated?)

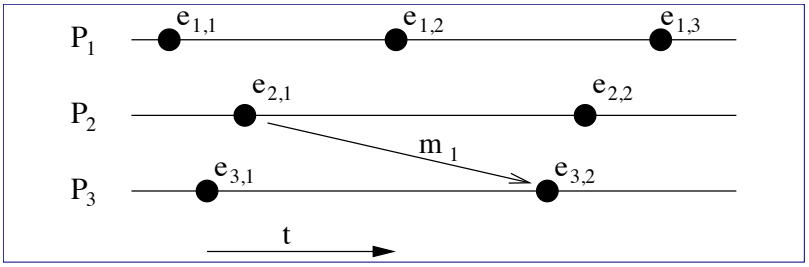
2.5 Total order with vector clocks

Lamport logical clocks create a partial order. This means that multiple events can have the same time stamp, which means that there is no unique order for these events. As seen in the lecture, you can extend this partial order to a **total order** by using tuples of Lamport clock and node ID (C_{P_k}, k), and defining a predicate "$<$" on two tuples in an appropriate way.

Now assume you have vector clocks instead of Lamport clocks. They also define a partial order. Explain how to define a **total order** on the basis of vector clocks!

2.6 Logical clocks – example

Consider this set of events and interactions between nodes:



a) Assign Lamport timestamps to each event, describe an algorithm for deriving a total order from Lamport timestamps (see also lecture), and sort all events according to this order.

b) Assign vector timestamps to each event, describe an algorithm for deriving a total order from these vector timestamps (see also Exercise 2.5), and sort all events according to this order.